

15

Sending POST, DELETE, and PUT HTTP Requests in Vue.js with Vuex

In the last chapter, we discussed state management and set up Vuex in our Vue.js app. We managed to write our first action and tried it in our application. However, the CRUD operations using Vuex in our application are not yet complete. In this chapter, we will write the remainder of the CRUD operations with HTTP requests and Vuex involvement.

With that, we will cover the following topics in this chapter:

- Removing a tour list using Axios and Vuex
- Adding a tour list using Axios and Vuex
- Using a non-async action in Vuex

- Removing a tour package using Axios and Vuex
- Adding a tour package using Axios and Vuex
- Updating a tour package using Axios and Vuex

Technical requirements

You will need Visual Studio Code to complete this chapter.

The finished repository of this chapter can be found at <https://github.com/PacktPublishing/ASP.NET-Core-and-Vue.js/tree/master/Chapter15/>.

Removing a tour list using Axios and Vuex

If you are still hungry to do some writing with Axios and Vuex, this is perfect for you. We will send a request to our ASP.NET Core Web API to retrieve the `TourList` collection, render them on the UI using Vuetify components, and then be able to delete any of the `TourList` objects. But before we start, let's update our ASP.NET Core Web API project.

Update the `TourPackageDto.cs` file in namespace `Travel.Application.Dtos.Tour` with the following code:

```
public float Price { get; set; }  
public string MapLocation { get; set; }
```

We are adding the `Price` and `MapLocation` properties.

Next, we update our backend service. Go to the `TourPackagesController.cs` file, which has a namespace of `Travel.WebApi.Controllers.v1`, and add a new async action method using the following code:

```
[HttpPut("{id}")]  
public async Task<ActionResult> Update(int id,  
    UpdateTourPackageCommand command)  
{  
    if (id != command.Id)  
        return BadRequest();  
    await Mediator.Send(command);  
    return NoContent();  
}
```

The preceding C# code is an additional controller for our Web API to edit an existing tour package in our application.

Now that we've updated our ASP.NET Core Web API project, let's focus on our Vue.js app and complete our Travel Tour web app's functionalities.

In my opinion, creating a delete functionality is the second easiest thing to implement next to the fetch functionality.

So, let's update the `DefaultContent.vue` file of the `AdminDashboard` directory. The first thing is to import `mapActions` into the `DefaultContent` component, like so:

```
import { mapActions } from "vuex";
```

We will also use `mapActions` in the `DefaultContent` component, just like in the `WeatherForecast` component in the previous chapter.

Next, we create the `methods` object and use `mapActions` like so:

```
methods: {  
  ...mapActions("tourModule", ["getTourListsAction"]),  
},
```

The preceding code gives us a `getTourListsAction` method.

Now create the `mounted` method, and let's call `getTourListsAction` inside it:

```
mounted() { this.getTourListsAction(); },
```

The preceding code will trigger `getTourListsAction` upon rendering the DOM.

Next, we write a localized style section and write a `default-content` class:

```
<style scoped>  
.default-content {  
  display: flex;  
  flex-direction: row;  
  flex-wrap: wrap;  
  justify-content: flex-start;  
}  
</style>
```

The preceding CSS will provide styling for a `div` tag that we will write in the next code.

Now let's update the `template` syntax section:

```
<template>
  <div>
    <div class="text-h2 my-4">Welcome to Admin
    Dashboard</div>
    <div class="default-content"></div>
  </div>
</template>
```

The preceding code is a simple welcome message that says `Welcome to Admin Dashboard`. We will come back to this component later after creating a `card` component.

Go to the `components` folder and create a `Vue.js` component file, `TourListsCard.vue`.

Add the following script to the newly created `Vue` component:

```
<script>
import { mapActions, mapGetters } from "vuex";
export default {
  name: "TourListsCard",
  computed: {
    ...mapGetters("tourModule", {
      lists: "lists", loading: "loading",
    }),
  },
};
</script>
```

We are bringing in `mapActions` and `mapGetters` from `Vuex`. We are then using `mapGetters` to bring the global states' `lists` and `loading` to local states' `lists` and `loading`. Take note that any changes in the global states' state and `loading` will also reflect in the localized `lists` and `loading` states.

Let's add the following code to the newly created `TourListsCard` `Vue` component:

```
<template>
  <v-skeleton-loader
    v-if="loading" width="300" max-width="450"
```

```

      height="100%" type="card" ></v-skeleton-loader>
    <v-card v-else width="300" max-width="450" height="100%">
      <v-toolbar color="light-blue" dark>
        <v-toolbar-title>Tour Lists</v-toolbar-title>
        <v-spacer></v-spacer>
      </v-toolbar>
      <v-list-item-group color="primary">
        <v-list-item v-for="tourList in lists"
          :key="tourList.id">
          <v-list-item-content>
            <v-list-item-title v-text="tourList.city"></v-
              list-item-title>
            <v-list-item-subtitle v-text="tourList.about">
              </v-list-item-subtitle>
          </v-list-item-content>
          <v-list-item-action>
            <div class="mr-2">
              {{ tourList.tourPackages &&
                tourList.tourPackages.length }}
            </div>
            <v-icon> mdi-delete-outline </v-icon>
          </v-list-item-action>
        </v-list-item>
      </v-list-item-group>
    </v-card>
  </template>

```

The preceding template syntax renders a loader and a card with `tourLists` packages. We are using the `v-for` directive to extract the objects from the lists and then render them one by one. Each `tourList` object will show the two values of two properties– the value of the city property and the value of the about property in the UI.

Now let's go back to `DefaultContent.vue` and update it.

Import the `TourListsCard` component from the `components` directory like so:

```
import TourListsCard from "@/components/TourListsCard";
```

You can use `@/` to navigate to the folders of the `src` directory easily. Now use the imported component as follows:

```
components: { TourListsCard, â€¦ },
```

We've registered the `TourListsCard` component in the components of `DefaultContent`.

Now, inside the `div` element with default content class, add the following code:

```
<div style="margin-right: 4rem; margin-bottom: 4rem">  
  <TourListsCard />  
</div>
```

We are rendering `TourListsCard` in the preceding code. Restart your server because we have created a new file.

In the `Travel.WebApi` project, run the following command:

```
dotnet run
```

The preceding command will run the whole application, which means both the backend and the frontend.

The following *Figure 15.1* shows a simple dashboard with a welcome message:

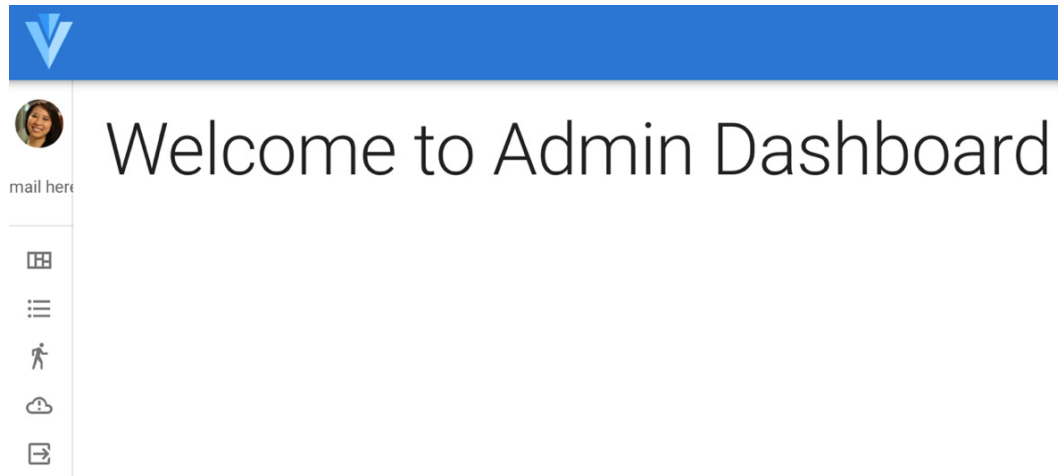


Figure 15.1 – A welcome message on the Admin Dashboard component

Now we are about to write the `delete` functionality in Axios and store it. You will find out how easy it is to write additional actions in our state management because we already set up our store in the last chapter. Now let's proceed.

Next, we update `services.js` of the `store/tour/` directory using the following code:

```
export async function deleteTourListAxios(id) {  
  return await api.delete("TourLists/" + id);  
}
```

The preceding code is a service that deletes a `tourList` object:

Let's also update `types.js` of `store/tour/`:

```
export const REMOVE_TOUR_LIST = "REMOVE_TOUR_LIST";
```

We are adding a new action type to the `types.js` file. The action type is for removing a tour list, hence the name `REMOVE_TOUR_LIST` in snake casing.

Let's also update the `actions.js` file of the `store/tour/` directory:

```
import { getTourListsAxios, deleteTourListAxios } from "@/  
store/tour/services";
```

We bring in the `deleteTourListAxios` service first before adding the `remove` function.

Now, let's write the `removeTourListAction` function like so:

```
// asynchronous action using Axios  
export async function removeTourListAction({ commit }, payload)  
{  
  commit(types.LOADING_TOUR, true);  
  try {  
    await deleteTourListAxios(payload);  
    commit(types.REMOVE_TOUR_LIST, payload);  
  } catch (e) {  
    alert(e);  
    console.log(e);  
  }  
  commit(types.LOADING_TOUR, false);  
}
```

The preceding function, which is `removeTourListAction`, takes a payload. The parameter you need to use when triggering `removeTourListAction` is technically just one, the payload, whereas the payload is an ID of `tourList` that we are going to remove.

Now, you will notice in `removeTourListAction` that it enables the loading by committing `types.LOADING_TOUR` and passing `true` before sending the delete request using the `deleteTourListAxios` service. The function also creates a commit that removes `tourList` and deletes it in ASP.NET Core.

`removeTourListAction` also commits `types.LOADING_TOUR` again with a Boolean of `false`, which will disable the loader component, or spinner component if you will, from spinning.

Now, update `mutations.js` of `store/tour/`:

```
[types.REMOVE_TOUR_LIST](state, id) {  
  state.lists = state.lists.filter((tl) => tl.id !== id);  
  state.packagesOfSelectedCity = [];  
},
```

The `types.REMOVE_TOUR_LIST` action type has an instruction to filter out all the objects by only getting an object with `id` that is not equal to `id` from the parameter. Then the action type also sets the `packagesOfSelectedCity` global state to an empty array.

Now, let's update the `TourListsCard.vue` component by adding a `methods` object with `mapActions` and an extra `removeTourList` function, which takes `listId` like so:

```
methods: {  
  ...mapActions("tourModule", ["removeTourListAction"]),  
  removeTourList(listId) {  
    const confirmed = confirm(  
      "You sure you want to delete this tour list? This  
      will also delete the packages permanently"  
    );  
    if (!confirmed) return;  
    this.removeTourListAction(listId);  
  },  
},
```


You will notice that there's a confirmation prompt before deleting or removing a tour list.

Now, let's update the `v-icon` component in the `template` syntax section of the `TourListsCard` Vue component like so:

```
<v-icon @click="removeTourList(tourList.id)">
  mdi-delete-outline
</v-icon>
```

In the preceding code, we are binding the `removeTourList` function to the `@click` directive, which will provide us with a click event.

If an ESLint error prevents the Vue.js app from refreshing, restart the server. Now rerun the ASP.NET Core Web API project by running the `dotnet run` command.

The command will start the whole application again. Now, open up your browser and head to the **Admin Dashboard** page of our application:

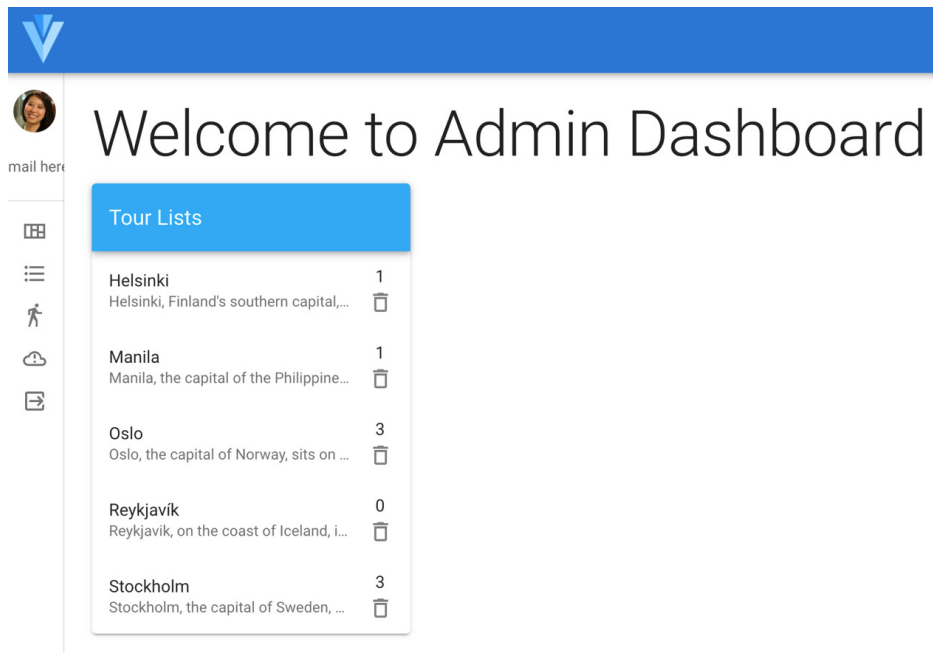


Figure 15.2 – Card component with the Tour Lists collection

The preceding *Figure 15.2* shows the card UI component after rerunning the application. You can now see the collection of `tourList` with the `delete` functionality. The numbers above the trash bin icon indicate packages available in a tour list. **0** means no package added as yet.

Try to delete **Manila** and look closely at the UI. The UI should update after removing a tour list.

We've successfully added the functionality that removes or deletes a tour list in our application. In the next section, we will create a form where we can create a new tour list.

Adding a tour list using Axios and Vuex

The Create or Add functionality would be the third easiest thing to implement in CRUD. Fetching and deleting functionalities is easy because they don't require a form component and some input fields. To create a functionality that adds or makes new data, we have to build a form.

Before building a form for the UI, we will first develop functionality in Axios and Vuex that creates a tour list.

To start, let's update `services.js` in `store/tour` by adding the following code:

```
export async function postTourListAxios(tourList) {  
  return await api.post("TourLists", tourList);  
}
```

The preceding code is a service that sends a *POST* request to the backend to create a `tourList` data entry.

Now, let's update `types.js` of the `store/tour` directory.

```
export const ADD_TOUR_LIST = "ADD_TOUR_LIST";
```

We are adding a new action type in the preceding code. The latest action type is for adding new `tourList`.

Next we update `actions.js` of the `store/tour` directory:

```
import { getTourListsAxios, deleteTourListAxios,  
  postTourListAxios, } from "@/store/tour/services";
```

Let's import the `postTourListAxios` service first before adding a new action.

After importing the newly created service, let's add a new action to `actions.js`:

```
// asynchronous action using Axios  
export async function addTourListAction({ commit }, payload) {  
  commit(types.LOADING_TOUR, true);
```

```

try {
  const { data } = await postTourListAxios(payload);
  payload.id = data; // storing the id from the response
  // int of ASP.NET Core, which will be used in the UI.
  payload.tourPackages = []; // initialize the
  // tourPackages of the newly created tourList
  commit(types.ADD_TOUR_LIST, payload);
} catch (e) {
  alert(e);
  console.log(e);
}
commit(types.LOADING_TOUR, false);
}

```

The preceding code is another asynchronous action function. It means that this is an action that uses Axios to send an HTTP request. The action takes `tourList` as the payload. It sends it to ASP.NET Core utilizing the service.

In response, the service returns `id` of the newly created tour list from the backend. We will then store the response in `payload.id` to use it in the mutation when we update our UI.

Now, let's update `mutations.js` in `store/tour` by adding the following code:

```

[types.ADD_TOUR_LIST](state, tourList) {
  state.lists.unshift(tourList);
},

```

The preceding mutation adds newly created `tourList` at the beginning of the array. The `unshift` helps us to see freshly added `tourList` in the card component easily.

Now we will create a helper function, and we will put it in a separate folder.

Let's create a new folder inside the `src` directory and name it `helpers`. Then, create a JavaScript file named `collections.js` inside the `helpers` folder.

The `collection.js` file has a `getCountryList` function that returns an array of countries, which we will need in the form's search input. We are going to create the form and bind the `getCountryList` helper function shortly.

So, get the helper function, called `getCountryList`, from the GitHub repository here: <https://github.com/PacktPublishing/ASP.NET-Core-and-Vue.js/tree/master/Chapter15>.

Now for the form. Create a Vue component named `AddTourListForm.vue` in the `components` folder and add the following script:

```
<script>
import { getCountryList } from "@helpers/collections";
import { mapActions } from "vuex";
export default {
  name: "AddTourListForm",
  data: () => ({
    bodyRequest: { city: "", country: "", about: "", },
    dialog: false,
    countryList: getCountryList(),
  }),
  methods: {
    ...mapActions("tourModule", ["addTourListAction"]),
  },
};
</script>
```

We are importing the `getCountryList` helper function as well as `mapActions` from Vuex. We declare a local state `bodyRequest` object, a Boolean `dialog`, and `countryList`, which is an array that `getCountryList` returns.

Now, the following code is for the template syntax of our `AddTourListView` component:

```
<template>
  <v-row justify="center">
    <v-dialog v-model="dialog" persistent max-
      width="600px">
      <template v-slot:activator="{ on, attrs }">
        <v-btn
          style="margin-top: 1rem" rounded color="light-
            blue"
            dark v-bind="attrs" v-on="on">
```

```

        <v-icon left>mdi-plus</v-icon>
        add new tour list
      </v-btn>
    </template>
    <!-- INSERT <v-card> BELOW -->
  </v-dialog>
</v-row>
</template>

```

`v-btn` is a button component from Vuetify, which will trigger the template with a `v-slot` directive activator to show up the dialog form later. A `v-slot` directive is used in wrapping another component to make it reusable.

You will notice that there's a comment that says insert `v-card`. We will add the following template syntax. Therefore, update `AddTourListForm.vue` with the following code:

```

<v-card>
  <form @submit.prevent="
    addTourListAction(bodyRequest);
    bodyRequest = {};">
    <v-card-title>
      <span class="headline">Create New Tour
      List</span>
    </v-card-title>
    <v-card-text>
      <v-container>
        <v-row>
          <v-col cols="12" sm="6">
            <v-text-field required label="City"
              v-model="bodyRequest.city"
            ></v-text-field>
          </v-col>
          <v-col cols="12" sm="6">
            <v-autocomplete required
              :items="countryList" label="Country"
              v-model="bodyRequest.country"
            ></v-autocomplete>

```

```
        </v-col>
      <!-- Removed for brevity -->
      <!-- Please see the full code in GitHub -->
      <v-btn color="blue darken-1" text
        @click="dialog = false" type="submit" >
        Save
      </v-btn>
    </v-card-actions>
  </form>
</v-card>
```

The preceding code is the form that will let us create new `tourList`. The form has a text field for `city`, a field with autocomplete for `country`, and a text area for `about`. We may notice that the `v-btn` component has a `submit` type. The `submit` type triggers @submit of the form that runs `addTourListAction`.

Now, let's include the form component in the `DefaultContent.vue` file by updating it:

```
import AddTourListForm from "@/components/AddTourListForm";
```

We are bringing in the `AddTourListForm` component from the components.

Next, we register `AddTourListForm` in the components object like so:

```
components: { TourListsCard, AddTourListForm, },
```

`AddTourListForm` is now ready to use in the template syntax area of the `Default` component.

Add the `AddTourListForm` component below the `TourListsCard` component:

```
<TourListsCard />
<AddTourListForm />
```

Restart the server because we have created a file. Run the `dotnet run` command to run the whole application.

Open your browser and go to the admin dashboard of the application, where you will see the following screenshot:

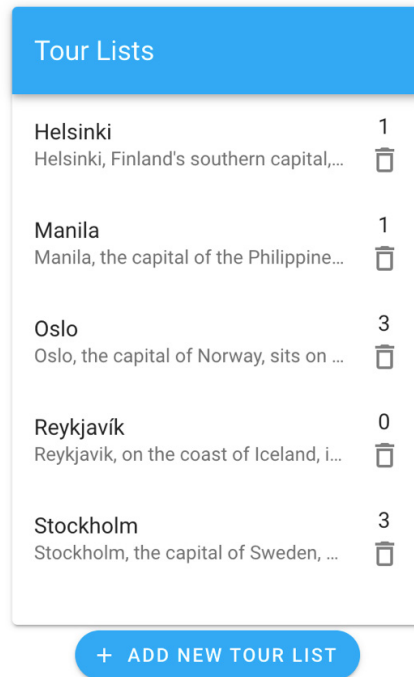


Figure 15.3 – ADD NEW TOUR LIST button

The preceding *Figure 15.3* shows the **ADD NEW TOUR LIST** button from Vuetify. Click it to open the form:

The screenshot shows a 'Create New Tour List' form. It has three input fields: 'City', 'Country' (with a dropdown arrow), and 'About'. Below the 'About' field is a text area. At the bottom left, there is a note '*indicates required field'. At the bottom right, there are two buttons: 'CLOSE' and 'SAVE'.

Create New Tour List

City Country

About

*indicates required field

CLOSE SAVE

Figure 15.4 – Adding a tour list Form

Figure 15.4 shows the form for creating or adding a new tour list. Try it and see the result of creating and sending a tour list.

In the next section, we will work on getting the tour packages for a particular tour list.

Using a non-async action in Vuex

We can see the collections of tour lists in the UI, but not their tour packages. This section will extract the tour packages from a tour list using Vuex without Axios because we are not sending an HTTP request. No HTTP request means that we are going to use a non-asynchronous action. You will learn how to use Vuex and a non-async action efficiently. So let's start.

Update the `types.js` file in the `store/tour` folder with the following code:

```
export const GET_PACKAGES_OF_SELECTED_CITY = "GET_PACKAGES_OF_SELECTED_CITY";
```

The preceding code is a new action type we are adding in `types.js`.

Then, update `actions.js` in `store/tour` with the following code:

```
// non-asynchronous action
export function getPackagesOfSelectedCityAction({ commit },
payload) {
  commit(types.GET_PACKAGES_OF_SELECTED_CITY, payload);
}
```

You will notice in the preceding action that there are no `try` and `catch` blocks since we are not using Axios in here.

Next, we update `state.js` of the `store/tour` folder:

```
packagesOfSelectedCity: [],
```

Finally, we have a state for our tour packages from a specific city.

Now, update `mutations.js` of the `store/tour` folder:

```
[types.GET_PACKAGES_OF_SELECTED_CITY](state, packages) {
  state.packagesOfSelectedCity = packages;
},
```


We are now using the payload as packages and then storing them in the `packagesOfSelectedCity` state, which is an array.

Next, we update `getters.js` of the `store/tour` folder:

```
packagesOfSelectedCity: (state) => state.
packagesOfSelectedCity,
```

We are going to use the preceding getter function for our UI later.

It's time to update the `TourListsCard` component. Go to the `TourListsCard.vue` file and update the component with the following code:

```
...mapActions("tourModule", [
  "removeTourListAction",
  "getPackagesOfSelectedCityAction",
]),
```

We are now exposing the new action, which is `getPackagesOfSelectedAction`.

Let's add a new function to `methods` using the following block of code:

```
addToPackages(packages, listId) {
  this.getPackagesOfSelectedCityAction(packages);
  this.$emit("handleShowPackages", true, listId);
},
```

We are using `getPackagesOfSelectedCityAction` and emitting `handleShowPackages`, which will act as props for the `TourListsCard` component.

Next, we update the `v-list-item-content` component of the `TourListsCard` component:

```
<v-list-item-content
  @click="addToPackages(tourList.tourPackages,
    tourList.id)">
```

The click event will trigger the `addToPackages` function that we just added in the `methods` block.

Now, let's update the `DefaultContent.vue` component file with the following code:

```
methods: {
  ...mapActions("tourModule", ["getTourListsAction"]),
```

```
handleShowPackages(show, listId) {  
  this.showPackages = show;  
  this.tourListId = listId;  
},  
},
```

We are adding a `handleShowPackages` method to the `methods` block.

Now, let's also add some local states:

```
data: () => ({ showPackages: false, tourListId: 0, } ),
```

We have `showPackages`, which is a Boolean type, and `tourListId`, which is a number.

Now let's use the `mounted` life cycle hook of the `DefaultContent.vue` component:

```
mounted() {  
  this.getTourListsAction();  
  this.showPackages = false;  
},
```

We call `getTourListsAction` and set `showPackages` to `false` whenever we go to the admin dashboard page.

Then, update `<TourListsCard />` in the template syntax:

```
<TourListsCard @handleShowPackages="handleShowPackages" />
```

We are using the emitted `handleShowPackages` props of `TourListsCard` and assigning the `handleShowPackages` function of `DefaultComponent.vue`.

Let's update the template syntax again with the following code:

```
<div style="margin-right: 4rem; margin-bottom: 4rem">  
  <TourListsCard @handleShowPackages  
    ="handleShowPackages" />  
  <AddTourListForm />  
</div>  
<div v-if="showPackages">  
  <h2>Tour Packages Card Here</h2>  
  <h3>Add Tour Package Form with tour list Id
```

```
Here</h3>
</div>
```

We are adding the `v-if` directive to the preceding template area code.

Now run the app and click any city.

The `TourPackagesCard` component should appear on the right side of the screen after running the app and clicking any city.

We are doing a quick proof-of-concept here that we can render the `TourPackagesCard` component, as shown in the following screenshot:

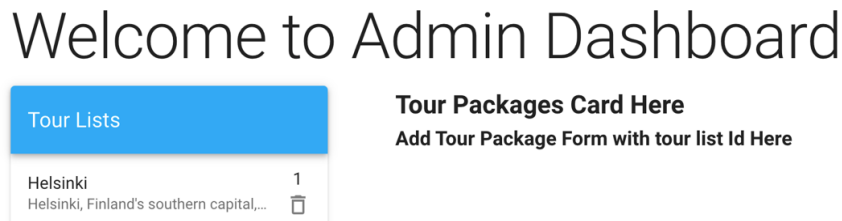


Figure 15.5 – Show placeholders

Figure 15.5 shows us that the placeholder messages pop up after selecting any **Tour Lists** card cities. Now we can conditionally show a UI. Let's create the card component for the packages.

Create `TourPackagesCard.vue` in the `components` folder and add the following script:

```
<script>
import { mapActions, mapGetters } from "vuex";
export default {
  name: "TourPackagesCard",
  computed: {
    ...mapGetters("tourModule", {
      packages: "packagesOfSelectedCity",
    }),
  },
};
</script>
```

The preceding script is importing and using `mapAction` and `mapGetters`.

Now let's add the `template` syntax section of `TourPackagesCard`. Add the following block of code:

```
<template>
  <v-container>
    <v-card width="500" max-width="600">
      <v-toolbar color="pink" dark>
        <v-toolbar-title>Packages</v-toolbar-title>
        <v-spacer></v-spacer>
      </v-toolbar>
      <v-list two-line>
        <v-list-item-group active-class="pink--text">
          <div v-if="packages && packages.length > 0">
            <template v-for="tourPackage in packages">
              <v-list-item :key="tourPackage.id">
                <!-- <template> for <v-list-item-content>
                here -->
              </v-list-item>
            </template>
            <v-divider />
          </div>
          <div v-else>
            <v-list-item>
              <v-list-item-content>
                <v-list-item-title
                  v-text="'No package added yet 🙄'"
                ></v-list-item-title>
              </v-list-item-content>
            </v-list-item>
          </div>
        </v-list-item-group>
      </v-list>
    </v-card>
  </v-container>
</template>
```

The preceding template code is a card component from Vuetify that renders every package of a selected city. The card component's look and feel will be similar to that of the tour lists, the only difference being the color.

Now let's add the list item content of the card component. Replace the comment `<!-- <template> for <v-list-item-content> here -->` with the following code:

```
<template>
  <v-list-item-content>
    <v-list-item-title v-text="tourPackage.name"></v-list-
      item-title>
    <v-list-item-subtitle
      class="text--primary"
      v-text="tourPackage.whatToExpect"
    ></v-list-item-subtitle>
    <div style="margin-top: 0.5rem; display: flex; flex-
      direction: row">
      <v-list-item-subtitle
        class="text--secondary"
        v-text="`Duration: ${tourPackage.duration}hrs`"
      ></v-list-item-subtitle>
      <v-list-item-subtitle
        class="text--secondary" v-text="
          `${tourPackage.instantConfirmation ? 'Instant
            Confirmation' : ''}`"
      ></v-list-item-subtitle>
    </div>
  </v-list-item-content>
  <v-list-item-action>
    <v-icon> mdi-delete-outline </v-icon>
  </v-list-item-action>
</template>
```

The preceding code is the UI for the row for each package. The row shows the values of the tour package's properties, such as name, whatToExpect, duration, and instantConfirmation.

Now again, update the `DefaultContent.vue` component with the following code:

```
import TourPackagesCard from "@/components/TourPackagesCard";
```

We are importing the `TourPackagesCard` component.

Let's register the `TourPackagesCard` component in our `components` block like so:

```
components: { TourListsCard, AddTourListForm,
  TourPackagesCard, },
```

Now, the `TourPackagesCard` component is available to use.

Replace the `v-if="showPackages"` and placeholder messages with the following code:

```
<div v-if="showPackages">
  <TourPackagesCard />
</div>
```

Now, our `TourPackagesCard` component should be visible after clicking a city.

Run the `dotnet run` command to start the whole application again.

Then, click on **Oslo**, which will show the `TourPackagesCard` component with some packages inside, as shown in the following screenshot:

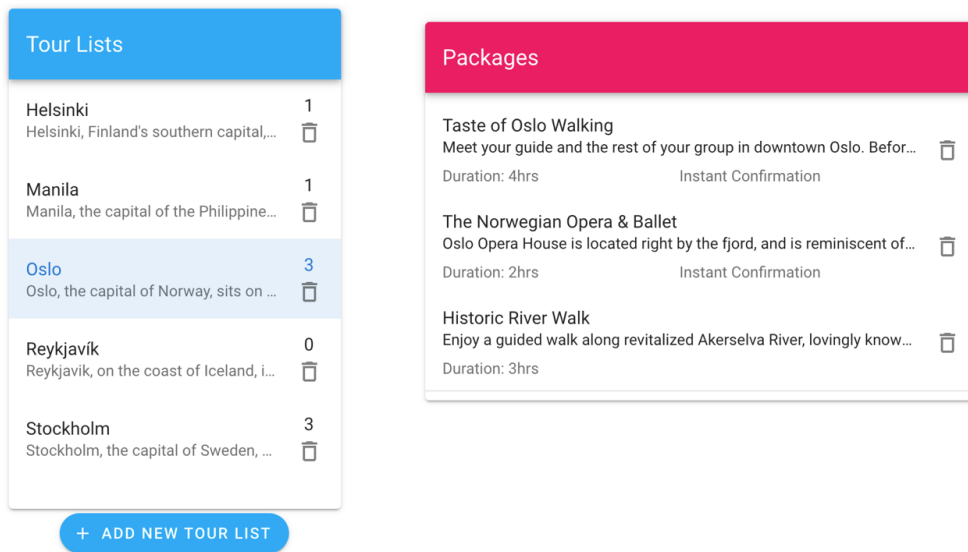


Figure 15.6 – `TourPackagesCard` component

Figure 15.6 shows the TourPackagesCard component on the right.

Now that we have finished the only non-async action in our application, let's go back to writing async actions in our Vuex.

Removing a tour package using Axios and Vuex

In this section, we will implement functionality that deletes a tour package in the backend and deletes the tour package in the UI. So let's start.

Update `services.js` of the `store/tour` folder:

```
export async function deleteTourPackageAxios(id) {
  return await api.delete("TourPackages/" + id);
}
```

We are adding a new service that deletes a tour package.

Next, we update `types.js` of the `store/tour` folder:

```
export const REMOVE_TOUR_PACKAGE = "REMOVE_TOUR_PACKAGE";
```

In the preceding code, we are adding a new action type to remove a tour package.

Next, we update `actions.js` of the `store/tour` folder:

```
import { getTourListsAxios, deleteTourListAxios,
  postTourListAxios, deleteTourPackageAxios,
} from "@/store/tour/services";
```

We are importing `deleteTourPackageAxios` in the preceding code.

Let's add another action to `actions.js`:

```
export async function removeTourPackageAction({ commit },
  payload) {
  commit(types.LOADING_TOUR, true);
  try {
    await deleteTourPackageAxios(payload);
    commit(types.REMOVE_TOUR_PACKAGE, payload);
  } catch (e) {
    alert(e);
  }
}
```

```
    console.log(e);  
  }  
  commit(types.LOADING_TOUR, false);  
}
```

The `removeTourPackageAction` function enables the loader to spin, sends a *DELETE* request to the backend using `Axios`, commits a type that will instruct the mutation to remove the UI tour package, and then disables the loader to stop it from spinning.

Then, update `mutations.js` in the `store/tour` folder:

```
[types.REMOVE_TOUR_PACKAGE](state, id) {  
  state.packagesOfSelectedCity =  
    state.packagesOfSelectedCity.filter(  
      (tp) => tp.id !== id;  
    ),  
}
```

The preceding mutation removes the UI tour package using a filter built into the JavaScript.

Now we can go to the `TourPackagesCard.vue` component and update it with the following code:

```
methods: {  
  ...mapActions("tourModule", ["removeTourPackageAction"]),  
  removeTourPackage(packageId) {  
    const confirmed = confirm(  
      "You sure you want to permanently delete this tour  
      package?");  
    if (!confirmed) return;  
    // might need to wait for 1 min because of the cache  
    this.removeTourPackageAction(packageId);  
  },  
},
```

We are adding a `methods` block and using `mapActions` to map `removeTourPackageAction`. We are then creating another method, called `removeTourPackage`, which triggers a confirm dialog and `removeTourPackageAction`.

And lastly, update the `v-icon` component by giving it a click event that runs the `removeTourPackage` function with `tourPackage.id` as an argument:

```
<v-icon @click="removeTourPackage(tourPackage.id)">
  mdi-delete-outline
</v-icon>
```

The `v-icon` component is ready to use. We can click it to remove a tour package. Go to the Vue app and click **City**. After clicking on a city, delete any tour package. You should see that the tour package gets removed from the UI.

Now we can go to the next section, which creates a functionality where we can add a new tour package.

Adding a tour package using Axios and Vuex

This section will build a functionality to add a new tour package to an existing tour list using Axios and Vuex.

Let's update `services.js` of the `store/tour` folder:

```
export async function postTourPackageAxios(tourPackage) {
  return await api.post("TourPackages", tourPackage);
}
```

In the preceding code, we add a new service that creates new `tourPackage` in the backend.

Next, we update `types.js` of the `store/tour` folder:

```
export const ADD_TOUR_PACKAGE = "ADD_TOUR_PACKAGE";
```

The code update adds a new action type for adding a tour package.

Now, update `actions.js` in `store/tour` using the following code:

```
import { getTourListsAxios, deleteTourListAxios,
  postTourListAxios, deleteTourPackageAxios,
  postTourPackageAxios,
} from "@/store/tour/services";
```

We import the service `postTourPackageAxios` first, as you can see in the preceding code. Now we can use imported `postTourPackageAxios` in the following code:

```
export async function addTourPackageAction({ commit }, payload)
{
  commit(types.LOADING_TOUR, true);
  try {
    const { data } = await postTourPackageAxios(payload);
    payload.id = data; // storing the id from the response
    // int of ASP.NET Core, which will be used in the UI.
    commit(types.ADD_TOUR_PACKAGE, payload);
  } catch (e) {
    alert(e);
    console.log(e);
  }
  commit(types.LOADING_TOUR, false);
}
```

`addTourPackageAction` turns on the loader, sends a *POST* request to the backend, adds `tourPackage` to the UI by calling a `commit`, and then turns off the loader.

Next, we update `mutations.js` of the `store/tour` folder:

```
[types.ADD_TOUR_PACKAGE](state, tourPackage) {
  state.packagesOfSelectedCity.unshift(tourPackage);
},
```

The preceding code is a mutation that adds the new `tourPackage` to the first index of the `packagesOfSelectedCity` array.

Then, for the form, where we can add new `tourPackage`, create a Vue component, `AddTourPackageForm.vue`, in `components` and the following script:

```
<script>
import { mapActions } from "vuex";
export default {
  name: "AddTourPackageForm",
  props: {
    tourListId: { type: Number, },
  },
```

```

data: () => ({
  id: 0,
  bodyRequest: {
    listId: 0, name: "", whatToExpect: "",
    mapLocation: "https://www.google.com/maps/place/...",
    price: 10, duration: 1, instantConfirmation: true,
  },
  dialog: false,
  currencies: ["USD", "NOK"],
  currencyValues: [0, 1],
  durations: [1, 2, 3, 4, 5, 6, 7, 8],
  durationValue: 1,
}),
methods: {
  ...mapActions("tourModule", ["addTourPackageAction"]),
  onSubmit() {
    this.bodyRequest.listId = this.tourListId;
    this.addTourPackageAction(this.bodyRequest); //
    triggers the method of the container holding this
    this.bodyRequest = {};
  },
},
};
</script>

```

We have `id`, `bodyRequest`, `dialog`, `currencies`, `currencyValues`, `durations`, and `durationValue` local states in the preceding code. We also have a `mapActions` spread in the methods and an `onSubmit` method that triggers `addTourPackageAction`. And lastly, the form has a `tourListId` props, which is `id` of `tourList`.

Add the following template syntax for the UI:

```

<template>
  <v-row justify="center">
    <v-dialog v-model="dialog" persistent max-
      width="600px">
      <template v-slot:activator="{ on, attrs }">

```

```

    <v-btn
      style="margin-top: 1rem" rounded color="pink"
      dark v-bind="attrs" v-on="on">
      <v-icon left> mdi-plus </v-icon>
      add new tour package
    </v-btn>
  </template>
  <!-- Insert <v-card> here -->
</v-dialog>
</v-row>
</template>

```

We have a `v-btn` component that opens up a dialog or modal form. This form is similar to the form of the tour lists.

Now, replace `<!-- Insert <v-card> here -->` with the following template syntax:

```

<v-card>
  <form @submit.prevent="onSubmit">
    <v-card-title>
      <span class="headline">Create New Tour
        Package</span>
    </v-card-title>
    <v-card-text>
      <v-container>
        <v-row>
          <v-col cols="12">
            <v-text-field required label="Name"
              v-model="bodyRequest.name"
            ></v-text-field>
          </v-col>
          <!-- Removed for brevity -->
          <!-- Please see the full code in GitHub -->
        </v-row>
      </v-container>
      <small>*indicates required field</small>
    </v-card-text>
  </form>
</v-card>

```

```

      <!-- Insert <v-card-actions> here -->
    </form>
  </v-card>

```

The preceding form lets the user add a new tour package. The form has the following fields for tour packages – name, whatToExpect, mapLocation, price, duration, and instant confirmation.

Let's also add some buttons:

```

<v-card-actions>
  <v-spacer></v-spacer>
  <v-btn color="blue darken-1" text @click="dialog =
false"> Close </v-btn>
  <v-btn color="blue darken-1" text @click="dialog =
false" type="submit">
    Save
  </v-btn>
</v-card-actions>

```

Close v-btn and Save v-btn both have functionalities that close the dialog, but Save v-btn is the submit type, which means it triggers @submit of for. @submit then triggers the onSubmit method from the methods block.

Next we update the DefaultContent.vue component again:

```

import AddTourPackageForm from "@/components/
AddTourPackageForm";

```

We are bringing in AddTourPackageForm. Let's then register AddTourPackageForm in the components block like so:

```

components: { TourListsCard, AddTourListForm,
  TourPackagesCard, AddTourPackageForm, },

```

We can now insert AddTourPackageForm in the template syntax section.

Insert the AddTourPackageForm component below the TourPackagesCard component. Don't forget to pass tourListId in the tourListId props like so:

```

<div v-if="showPackages">
  <TourPackagesCard />

```

```
<AddTourPackageForm :tourListId="tourListId" />  
</div>
```

AddTourPackageForm is now in the `template` syntax and ready to use.

If an ESLint error is preventing the Vue.js app from refreshing, restart the server.

OK, so let's check out the browser to see the **ADD NEW TOUR PACKAGE** button, as shown in the following screenshot:

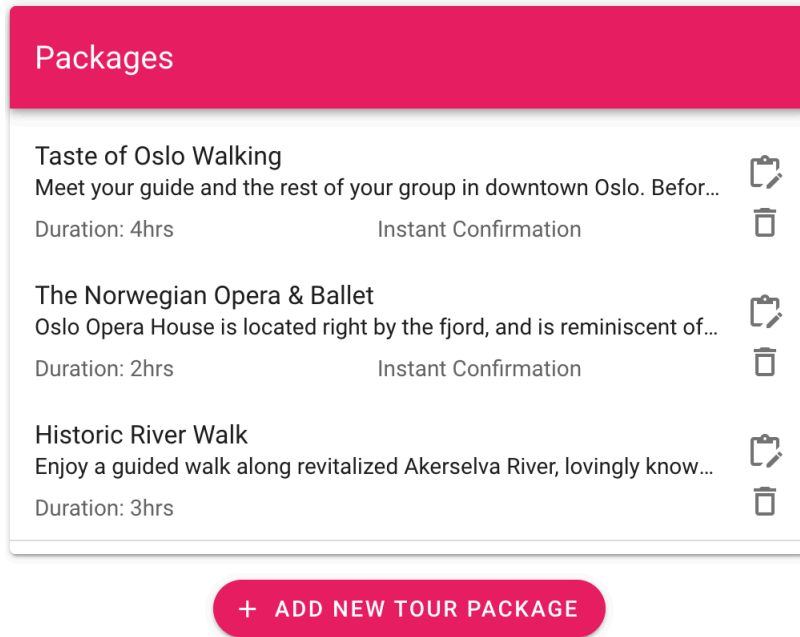


Figure 15.7 – ADD NEW TOUR PACKAGE button

Now, click the new button to see whether the dialog form will appear, as shown in the following screenshot:

Figure 15.8 – Create New Tour Package form

Figure 15.8 shows the form for creating a new tour package. Try to fill out the form and hit **SAVE**.

Now we can move on to the next section.

Updating a tour package using Axios and Vuex

We are in the last section of this chapter, and this section is about updating a tour package using a *PUT HTTP* request and Vuex. We are going to create another form for editing a tour package. So let's start.

The first task is to update `services.js` in the `store/tour` directory:

```
export async function putTourPackageAxios(tourList) {
  return await api.put(`TourPackages/${tourList.id}`,
    tourList);
}
```

We are adding a new service that sends a *PUT* request to our backend server.

Next, we update `types.js` of the `store/tour` folder again:

```
export const UPDATE_TOUR_PACKAGE = "UPDATE_TOUR_PACKAGE";
```

The preceding code is a new action type for updating a tour package.

Next, we also update `actions.js` in the `store/tour` folder:

```
import { getTourListsAxios, deleteTourListAxios,
  postTourListAxios, deleteTourPackageAxios,
  postTourPackageAxios, putTourPackageAxios,
} from "@/store/tour/services";
```

In the preceding code, we are importing the `putTourPackageAxios` service.

The following code is a new asynchronous Vuex action. The action will be the last Vuex action in the `actions.js` file:

```
export async function updateTourPackageAction({ commit },
payload) {
  commit(types.LOADING_TOUR, true);
  try {
    await putTourPackageAxios(payload);
    commit(types.UPDATE_TOUR_PACKAGE, payload);
  } catch (e) {
    alert(e);
    console.log(e);
  }
  commit(types.LOADING_TOUR, false);
}
```

The newly added action called `updateTourPackageAction` enables the loader, sends a *PUT* request with a payload to our backend, and commits `UPDATE_TOUR_PACKAGE`, passing the payload to also update the UI, and then disables the load in the UI.

Then we also update `mutations.js` in `store/tour` with the following code:

```
[types.UPDATE_TOUR_PACKAGE](state, payload) {
  const packageIndex =
    state.packagesOfSelectedCity.findIndex(
```



```

      (pl) => pl.id === payload.id);
    state.packagesOfSelectedCity[packageIndex] = payload;
    const listIndex = state.lists.findIndex(
      (l) => l.id === state.packagesOfSelectedCity.listId);
    state.lists[listIndex] = state.packagesOfSelectedCity;
  },

```

We are adding a new mutation that finds the index of the specific tour package. After finding the index, we locate the particular tour package via the index and then mutate it with the payload, the updated tour package.

We also update `tourList` by again finding the index of `selectedCity` and then editing it. The mentioned logic keeps all the affected objects and arrays in sync.

Now, create a Vue component named `UpdateTourPackageForm.vue` in the `components` directory and add the following script to the new Vue component:

```

<script>
import { mapActions } from "vuex";
export default {
  name: "UpdateTourPackageForm",
  props: {
    bodyRequest: {
      type: Object,
      required: true,
      default: { listId: 0, name: "missing name",
        whatToExpect: "missing what to expect",
        mapLocation: "missing map location",
        price: 0, duration: 0, instantConfirmation: true,
      },
    },
  },
  methods: {
    ...mapActions("tourModule",
      ["updateTourPackageAction"]),
    onSubmit() {
      this.updateTourPackageAction(this.bodyRequest); //
      fyi, you might not see the results right away because
      of the cache.
    }
  }
}

```

```
    },  
  },  
  data: () => ({  
    dialog: false,  
    currencies: ["USD", "NOK"],  
    currencyValues: [0, 1],  
    durations: [1, 2, 3, 4, 5, 6, 7, 8],  
    durationValue: 1,  
  }),  
};  
</script>
```

The preceding script provides a `bodyRequest` props, uses `mapActions`, has the `onSubmit` method that invokes `updateTourPackageAction`, and a set of local states – namely `dialog`, `currencies`, `currencyValues`, `durations`, and `durationValue`.

Now, let's add the template syntax using the following code:

```
<template>  
  <v-row justify="center">  
    <v-dialog v-model="dialog" persistent max-  
      width="600px">  
      <template v-slot:activator="{ on, attrs }">  
        <v-icon class="mr-3" v-bind="attrs" v-on="on">  
          mdi-clipboard-edit-outline  
        </v-icon>  
      </template>  
      <v-card>  
        <form @submit.prevent="onSubmit">  
          <v-card-title>  
            <span class="headline">Update Tour  
              Package</span>  
          </v-card-title>  
          <v-card-text>  
            <!-- Insert <v-container> Here -->  
            <small>*indicates required field</small>  
          </v-card-text>
```

```

    <v-card-actions>
      <v-spacer></v-spacer>
      <v-btn text color="blue darken-1"
        @click="dialog = false">
        Close
      </v-btn>
      <v-btn text color="blue darken-1"
        @click="dialog = false" type="submit">
        Update
      </v-btn>
    </v-card-actions>
  </form>
</v-card>
</v-dialog>
</v-row>
</template>

```

The preceding block of template syntax is the UI for `UpdateTourPackageForm.vue`. The template uses the `v-card` component, which looks neat, and the form that has the `@submit` directive event to invoke or trigger the `onSubmit` function.

Next, we replace `<!--Insert <v-container> Here -->` with the following template syntax:

```

<v-container>
  <v-row>
    <v-col cols="12">
      <v-text-field required label="Name"
        v-model="bodyRequest.name"
      ></v-text-field>
    </v-col>
    <v-col cols="12">
      <v-textarea
        label="What to expect"
        v-model="bodyRequest.whatToExpect" required
      ></v-textarea>
    </v-col>
  <!-- Removed for brevity -->

```

```

    <!-- Please see the full code in GitHub -->
  </v-col>
</v-row>
</v-container>

```

The preceding template is responsible for providing the fields needed in the form. These tour package fields are as follows: name, whatToExpect, mapLocation, price, duration, and instantConfirmation. You will notice that the fields are using the bodyRequest props because those props will be tourPackage the user will be editing.

Now, let's update TourPackagesCard.vue with the following code:

```

import UpdateTourPackageForm from "@/components/
UpdateTourPackageForm";

```

We are importing UpdateTourPackageForm into TourPackagesCard.vue. We will then register UpdateTourPackageForm in the components block like so:

```

components: { UpdateTourPackageForm },

```

We can now use UpdateTourPackageForm in the template syntax area.

Update the <v-list-item-action> component in the template section with the new following code:

```

<v-list-item-action>
  <UpdateTourPackageForm :bodyRequest="tourPackage" />
  <v-icon @click="removeTourPackage(tourPackage.id)">
    mdi-delete-outline
  </v-icon>
</v-list-item-action>

```

The preceding block of template code includes the UI for updating a tour package.

Now, before testing out the new form, let's update our ASP.NET Core Web API first. Go to UpdateTourPackageCommandValidator.cs of the Travel.Application.TourPackages.Commands.UpdateTourPackage namespace. Then, edit the maximum length rule for Name like so:

```

public UpdateTourPackageCommandValidator(IApplicationDbContext
context)
{
    _context = context;
}

```

```

RuleFor(v => v.Name)
    .NotEmpty().WithMessage("Name is required.")
    .MaxLength(200).WithMessage("Name must not
exceed 200 characters.");
}

```

The name should only be 200 characters. However, this is not enough. We will also create a validation in the frontend, but this will be covered in *Chapter 17, Input Validations in Forms*.

Now rerun the app to see whether the application is still running without any problems:

Globen Skyview

This touristy trip will give you a good view of the city. It's a little on t...

Duration: 2hrs

Instant Confirmation



Figure 15.9 – Pencil pad icon

Figure 15.9 shows a piece of a tour package. Try to update a tour package by clicking the pencil pad icon, which will open up the `UpdateTourPackageForm` component.

Now, let's summarize what you have learned in this long chapter about HTTP requests, Vuex state management, and forms from Vuetify.

Summary

And you have finished. Good work! Now let's break down what you've gained from this chapter. You have learned how to do CRUD using Axios, an HTTP client library. You have learned how to write asynchronous and non-asynchronous actions in Vuex. You have also learned how to use `mapActions` and `mapGetters` in components. And lastly, you have learned how to use Vuetify to build attractive forms and buttons.

In the next chapter, we will build the login and sign-up forms for our users. We will also enable authorization in our backend so that only authenticated users can send CRUD operations to our ASP.NET Core Web API.